

Department of Computer Science and Engineering

Course Code: CSE341	Credits: 1.5
Course Name: Microprocessors	Semester: Summer'26

Lab 04

Addressing Modes

I. Topic Overview:

This laboratory session explores the seven fundamental data addressing modes of the Intel 8086 microprocessor and the implementation of program control flow through jump, call, and return instructions. Students will develop the technical proficiency required to trace data traversal through registers and memory locations. To ensure mastery of low-level memory logic, the use of complex data structures such as arrays is strictly prohibited in this lab.

II. Lesson Fit:

Prerequisites for this lab include foundational knowledge from Labs 01-03:

- **8086 Register Sets:** Comprehensive understanding of General (AX, BX, CX, DX), Pointer (SP, BP), Index (SI, DI), and Segment (CS, DS, SS, ES) registers.
- **EMU8086 Environment:** Familiarity with the simulator for writing, compiling, and single-stepping assembly code.
- **Basic Instruction Set:** Proficiency with MOV, ADD, and SUB operations.

III. Learning Outcome:

Upon completion of this lab, students will be able to:

- Differentiate and implement all seven primary data addressing modes.
- Execute unconditional jumps and implement subroutines using CALL and RET.

- Trace the movement of data across various memory addressing schemes.
- Analyze the functional impact of the Clear Carry (CLC) and Halt (HLT) instructions.

IV. Technical Discussion: Data Addressing Modes

Data addressing modes specify how the 8086 microprocessor identifies the operands for its instructions.

1. **Immediate Addressing:** The data is a constant value contained within the instruction.
2. **Register Addressing:** The data is located within an internal CPU register.
3. **Direct Addressing:** The data is found at a specific memory offset provided directly in the instruction.
4. **Register Indirect Addressing:** The memory offset is stored in a register (BX, BP, SI, or DI).
5. **Base-plus-index Addressing:** The offset is calculated by adding a base register (BX/BP) and an index register (SI/DI).
6. **Register Relative Addressing:** The offset is determined by adding a displacement (constant) to a register.
7. **Base-relative-plus-index Addressing:** The offset is the sum of a base register, an index register, and a displacement.**Note on Segment Defaults:** Based on Lab 01 standards, memory access using the BP register defaults to the **Stack Segment (SS)** . Conversely, memory access via BX, SI, or DI defaults to the **Data Segment (DS)** .

Instruction Note

- **CLC (Clear Carry Flag):** Explicitly resets the Carry Flag (CF) to 0.
- **HLT (Halt):** Stops the execution of the microprocessor.

V. Technical Discussion: Control Flow

Unconditional Jump (JMP)

The JMP instruction facilitates a direct transfer of program control to a specified label. This mechanism allows the processor to skip intermediate instructions, enabling the implementation of non-sequential logic and infinite loops without relying on status flags.

Procedures (CALL/RET)

Procedures allow for modular, reusable code blocks.

- **CALL:** The microprocessor pushes the current return address onto the stack and updates the Instruction Pointer (IP) to the procedure's entry point. Following the stack's LIFO (Last-In, First-Out) nature, the **Stack Pointer (SP) decrements by 2** during this operation.
- **RET:** This instruction pops the return address from the stack back into the IP. Consequently, the **Stack Pointer (SP) increments by 2** .
- **Stack Mechanism:** The stack is a top-down structure. As data is pushed, the pointer decrements toward lower memory addresses.

VI. Activity Detail:

- **Hour 1:** Discussion on the technical mechanics and syntax of the seven Addressing Modes.
- **Hour 2:** Analysis of Control Flow (JMP vs. CALL/RET) and Tracing Table demonstration.
- **Hour 3:** Independent execution of Lab Tasks and individual Viva evaluation.

VII. Home Tasks

- **Task 1:** Write a program to capture a single character input from the user (INT 21H / AH=1). Store this character in a **byte-sized variable (DB)** using **Direct Addressing** , then move the value from memory into the BX register using **Register Indirect Addressing** .
- **Task 2:** Store two word-sized values in memory. Utilize **Base-plus-index addressing** to perform addition on these two values and store the result in the AX register.
- **Task 3:** Define a variable in the data segment. Implement a **Register Relative** move where a displacement of 5 is used to access the variable and load it into a register.
- **Task 4:** Write a program that initializes CX with a specific value. Implement a JMP instruction to skip a block of code designed to decrement CX, thereby "protecting" the original value from modification.

Trace the value of register and identify which addressing mode:

Line	Instruction	Addressing Mode	Registers Value
1	MOV AX, 1234h		
2	MOV [5000h], AX		
3	MOV DI, 02h		
4	MOV BX, [DI+5000h]		
5	MOV BX, 5000h		
6	MOV CX, [BX+02h]		
7	MOV DL, [5000h]		
8	MOV DH, [5001h]		
9	MOV SI, [5000h]		

10	MOV BX, TARGET2		
11	JMP TARGET1		
12	MOV DX, 5555h		
—	TARGET1:		
13	JMP BX		
14	MOV DX, 7777h		
—	TARGET2:		
15	MOV CX, 0AAAAh		